별무리: 온디바이스 AI 에이전트 앱

AI·SW 마에스트로 서울 제17기
2026.04 ~ 2026.11 (정식 출시 예정) / 온디바이스 AI 개발

소스 코드: https://github.com/mornye-minor-gallery/PetAI-AI

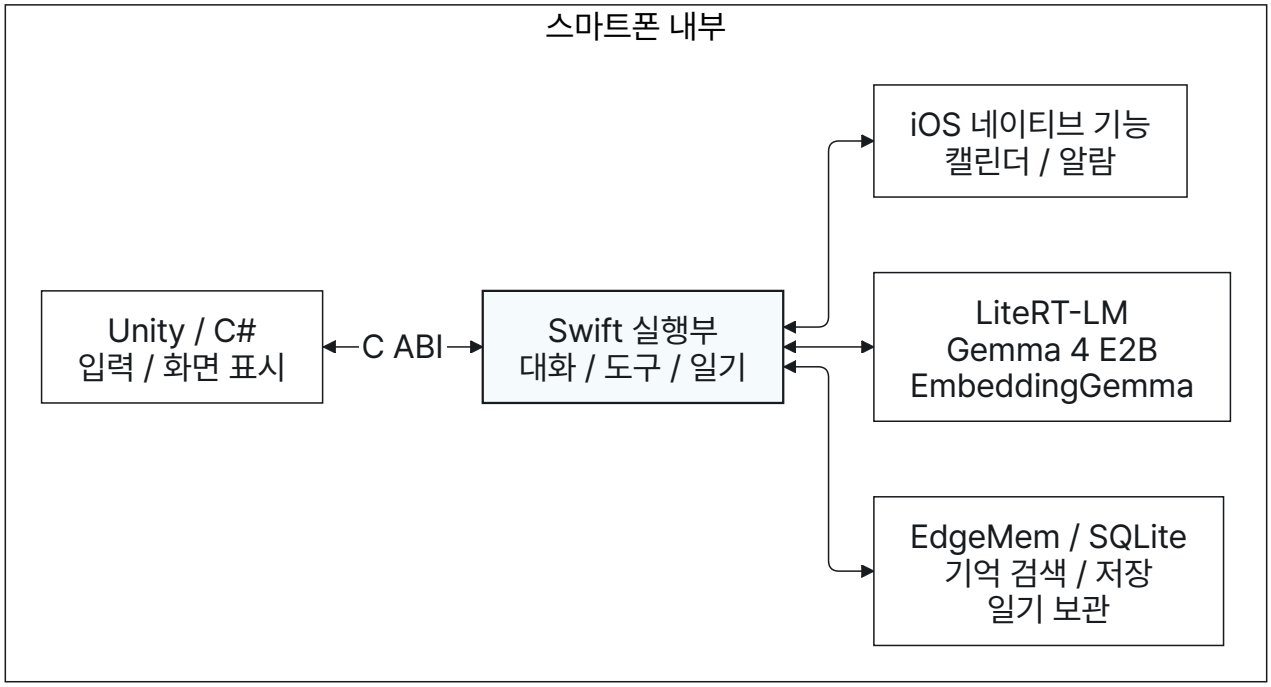
프로젝트 개요

- 별무리는 스마트폰에서 완전히 온디바이스로 실행되는 소형 언어 모델(sLM)로 캐릭터와 대화하는 에이전트 앱입니다. 그날의 중요한 기억을 매일 일기로 저장하며, 자연어로 캘린더 정리와 알람 설정 등의 네이티브 기능을 호출하여 작업하는 Agent 기능을 지원합니다. 완전 온디바이스는 서비스 비용과 개인정보 문제 때문에 택했고, 한국어 대화가 불안정한 다른 소형 모델들 대신 팀원 블라인드 테스트에서 가장 좋았던 Gemma 4 E2B를 골랐습니다.

담당 범위

- Effective Parameter 2.3B의 파라미터 규모를 가진 소형 언어 모델 Gemma 4 E2B 모델을 활용하여 완전한 모바일 (Android, iOS) 온디바이스 추론을 구현했습니다.
- KVCache 등의 최적화를 위해 Google의 오픈소스 엣지 디바이스 추론 엔진인 LiteRT-LM을 프로젝트에 맞게 직접 수정하고 적용했습니다.
- 300M의 파라미터 규모를 가진 EmbeddingGemma를 활용한, 개인화 소형 언어 모델을 위한 장기 기억 아키텍처 EdgeMem을 직접 설계하고 적용했으며, iOS와 Android 네이티브 기능 Tool Calling을 구현했습니다.
- 게임 클라이언트 Unity와 LiteRT-LM 추론 엔진 실행부를 책임지는 Swift 엔진을 Application Binary Interface로 통신하여 C#과 C++ 두 언어의 동작을 통합했습니다.

전체 구조

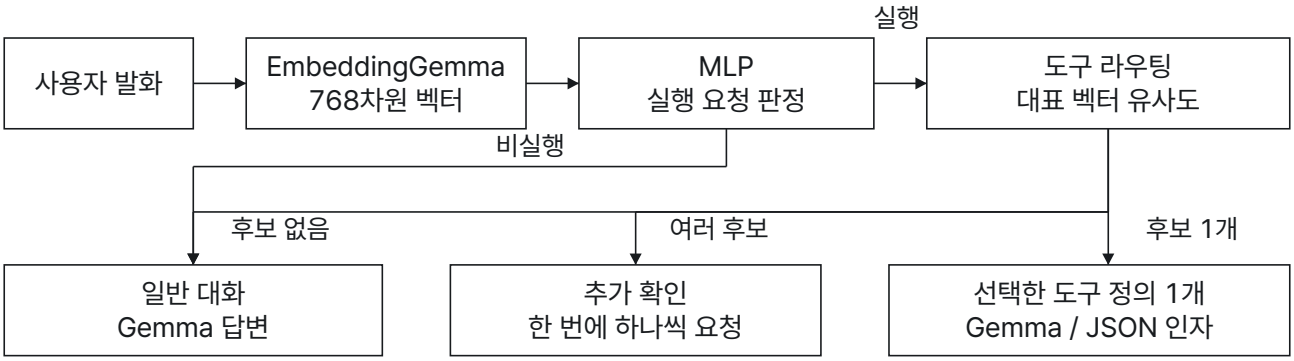


- Unity 클라이언트는 C ABI로 사용자 입력을 Swift 실행부에 전달하고, 답변과 실행 결과를 받아 화면에 표시합니다. Swift는 대화와 도구 요청을 구분하며, LiteRT-LM의 모델 실행, EdgeMem의 기억 검색과 저장, iOS 네이티브 기능 호출을 처리합니다. 일기는 그날 저장한 기억으로 생성해 기기에 보관합니다.

사용 기술

클라이언트	Unity / C#
네이티브 실행부와 연동	Swift / C/C++ / C ABI
추론 엔진	LiteRT-LM
모델	Gemma 4 E2B / EmbeddingGemma
기억 저장	SQLite

1. 일반 대화의 도구 호출 오인을 줄인 실행 의도 분류



문제

- 개발 중 테스트에서 일반 대화가 도구 요청으로 잘못 분류되는 경우를 반복해서 확인했고, 이대로는 실제 앱에서 쓸 수 없겠다고 판단했습니다.
- "내가 만든 알람 목록을 확인하는 기능도 지원해?"처럼 도구와 표현이 비슷하지만 실행 요청이 아닌 문장(Hard Negative)이 특히 어려웠습니다. Gemma에 직접 판정을 맡기더라도 이런 False Positive가 많았습니다.
- UX 관점에서는 실제 요청을 놓치는 것보다 원하지 않는 기능이 실행되는 것이 사용자에게 더 큰 불쾌감과 불편함을 준다고 생각했습니다.

해결 과정

- Regex 같은 키워드 규칙으로는 같은 단어가 들어간 기능 문의와 실행 요청을 구분하기 어렵습니다. EmbeddingGemma가 발화의 Representation을 충분히 뽑아낸다고 생각했고, **작은 분류 헤드만 있으면 두 유형이 Separable할 것이라는 가설**을 세웠습니다.
  - 768차원 벡터를 입력받는 MLP로 실행 의도를 판정하고, 통과한 발화의 같은 벡터로 도구를 선택했습니다.
- SLM은 작업이 길어질수록 정확도가 떨어지고 추론도 오래 걸리기 때문에 최대한 덜 맡겨야 한다고 생각했습니다. 그래서 Gemma에는 선택된 도구 하나의 정의를 주고 JSON 인자 생성만 맡겼습니다.
  - Gemma가 만든 인자는 Swift에서 형식과 허용 값을 검증하고, 사용자가 승인한 뒤에만 기능을 실행했습니다.
- Hard Negative를 학습 데이터에 추가했습니다.
  - 한국어 Hard Negative만 보강하자 일반 대화 오인은 줄었지만 실제 실행 요청도 많이 놓치는 문제가 발생했습니다.
  - 그래서 Hard Negative 비실행 문장과 LLM 후처리로 생성한 실행 문장을 Pair로 사용하여 데이터를 증강했습니다.

결과

학습 구성별 분류 결과 (기존 평가셋, 실행 요청 168건과 일반 대화 24건)

학습 구성	대화 오인율 ↓	요청 탐지율 ↑
임베딩 유사도만 (Baseline)	45.8%	96.4%
Gemma 직접 판정	37.5%	**100.0%**
초기 MLP	66.7%	97.0%
한국어 Hard Negative만 보강	8.3%	70.2%
실행/비실행 함께 보강	8.3%	95.2%
위 구성 + 한국어 Hard Negative	**4.2%**	94.6%

- 일반 대화의 도구 요청 오인을 임베딩 Baseline 대비 **90.9%** 줄였습니다(**11/24**건 → **1/24**건).
- **Trade-Off:** 마지막 두 구성의 정답 수가 같아(182/192), 실제 요청을 한 건 더 놓치더라도 원하지 않는 실행이 적은 쪽을 선택했습니다.
- 구성 선택에 쓰지 않은 신규 평가 280문항에서도 False Positive를 **19.3%에서 2.9%**로 줄였고, 단일 도구 선택률은 94.3%에서 90.0%로 낮아졌습니다.

2. 온디바이스 추론 최적화

문제

- TestFlight 베타 사용자들이 공통적으로 답변이 늦게 나온다는 피드백을 주었습니다. 디코드 속도는 괜찮았지만 첫 응답까지 기다리는 시간(TTFT)이 길게 느껴져, 프리필이 느리다고 판단했습니다. 매 요청에는 앱 전용 고정 프롬프트가 포함되어 매번 다시 계산되고 있었습니다.
- Context Length 어블레이션 실험 중, Context Length를 늘리자 메모리 사용량이 예상보다 훨씬 크게 늘었습니다.

해결 과정

- 처음에는 메모리의 KV 캐시만 재사용하려고 했습니다.
  - CachedSession으로 메모리의 KV 캐시를 유지하고, 같은 고정 프롬프트를 다음 요청에 재사용했습니다.
  - 고정되는 프롬프트를 위에, 매 턴 바뀌는 퓨샷 예시와 기억, 대화는 아래에 배치했습니다. 이전 입력과 token ID를 대조해 완전히 같은 prefix까지만 캐시를 재사용하고, 나머지는 모두 프리필했습니다.
- 그런데 생각해 보니 앱을 종료하면 메모리의 캐시가 사라져, 다시 실행한 뒤에는 같은 프롬프트를 처음부터 계산해야 했습니다. 엔진 공개 C API에는 추론 상태를 꺼내고 넣는 기능이 없었기에, **LiteRT-LM C++ 엔진을 포크해 CPU 세션 상태 전송 API를 직접 추가**했습니다.
  - 엔진은 KV 텐서, 처리 위치, 토큰 ID, 샘플러 상태의 전송만 맡고, 파일 저장과 복원, 모델 정보와 체크섬 검사는 Swift가 맡도록 나눴습니다.
- 메모리는 KV 캐시처럼 Context Length에 비례해 늘 것이라고 예상했지만, heap과 malloc_history로 할당 스택을 추적해 보니 CPU 연산 라이브러리 XNNPACK에서 그렇지 않은 버퍼가 나왔습니다.
  - Context Length 2048, 4096, 8192에서 16, 64, 256MiB로 4배씩 커지는 버퍼 9개를 찾았습니다. 4096에서 8192로 늘릴 때 최고 메모리 증가분 1,827MiB 중 1,728MiB(약 95%)가 이 버퍼였습니다.
  - 버퍼 크기는 채널 × batch × 4B로 계산되는데, **채널 수와 batch가 모두 Context Length에 비례해 Context Length가 두 번 곱해지고 있었습니다.**
  - 배치 차원 계산을 수정해 버퍼 하나를 64MiB에서 16KiB로 줄이고(Context Length 4096), 수정 전후를 비교했습니다.
- 테스트
  - iPhone 17에서 같은 대화를 캐시 끔과 켬으로 반복 실행해, TTFT와 함께 답변이 그대로인지 비교했습니다.
  - 앱을 직접 종료하고 다시 실행하며 캐시 저장과 복원 동작을 확인했고, 다른 프로세스에서 복원한 출력이 처음부터 계산한 출력과 같은지도 비교했습니다.
  - 메모리 수정도 수정 전후의 최고 메모리와 답변을 함께 비교했습니다.

결과

메모리 KV 캐시 재사용 전후의 TTFT

측정 구간	캐시 재사용 전	캐시 재사용 후
전체 평균	4.920초	1.274초
후속 요청 평균	4.872초	0.446초

- 메모리 KV 캐시 재사용으로 TTFT를 전체 평균 **74.1%**, 후속 요청 평균 **90.8%** 줄였습니다. 비교한 답변 15쌍은 모두 일치했습니다.
- 할당 수정 후 iPhone 17의 최고 메모리를 1.866GiB에서 1.063GiB로 약 **0.8GiB** 줄였습니다.

3. 퓨샷 예시 검색의 쿼리 선택

문제

- Gemma 4 E2B는 대화 표현력이 낮아 문맥에 맞는 반응을 이끌어 내기 어려웠습니다. 그런데 적절한 예시를 직접 넣어 주면 매우 자연스럽게 대답했습니다.
- 모델을 파인튜닝하는 방법도 있었지만, 학습할 때마다 모바일용 모델로 다시 변환해 앱에 배포해야 했습니다. 반면 **예시는 모델을 건드리지 않고 풀만 계속 늘리면 되므로 확장성이 유리하다고 판단해 퓨샷 방식을 선택했습니다.**
- 로컬에서 실행한 Gemma 4 31B를 교사 모델로 캐릭터의 반응 예시를 미리 만들고, EmbeddingGemma로 비슷한 예시를 찾아 퓨샷으로 넣었습니다.
- **Trade-Off:** 예시가 Context 자리를 차지해 프리필이 늘어나는 비용은 감수했습니다.
- 하지만 예시를 제공해도 실제 앱에서는 출력이 잘못되는 상황이 많았습니다. 임베딩 기반의 검색 결과를 관찰하니 상황에 맞지 않는 예시가 들어가 있었고, **핵심 문제가 모델보다 예시를 찾는 검색에 있다고 판단했습니다.**

해결 과정

- 원인을 분석해 보니 **발화 유형에 따라 필요한 검색 쿼리가 달랐습니다.**
  - 새 화제를 꺼낸 말(Topical)은 현재 문장만으로 검색해야 정확했습니다.
  - 앞 대화에 기대는 말(NonTopical)은 이전 대화까지 쿼리로 주어야 맞는 예시가 나왔습니다.

표 1. 발화 유형별 검색 관련성 (검증셋, 2점 만점)

발화 유형	현재 문장만	이전 대화 포함
Topical (120개)	**1.017**	0.633
NonTopical (120개)	0.508	**0.733**

- 그래서 턴마다 발화 유형을 판정해 검색 입력을 바꾸는 실행 로직을 제안했습니다.
- 판정 방법은 다음 기준으로 비교했습니다.
  - Codex로 합성 대화 800개를 만들고, 발화 유형 정답은 작성과 분리한 별도 요청이 원문을 보고 검수한 뒤 학습셋 390개, 검증셋 240개, 테스트셋 160개로 나눴습니다.
  - 맞는 예시에는 정해진 정답이 없었기에, 검색 결과의 관련성은 블라인드 LLM-as-a-judge로 0~2점 채점했습니다.
  - Rule 기반 판정, EmbeddingGemma Prototype 유사도, 문자 n-gram TF-IDF 유사도, 이들의 Weighted Voting을 검증셋에서 비교했습니다. 검증셋 1등(97.1%)은 턴마다 임베딩을 2회 계산해야 했습니다. 현재 문장으로 먼저 판정하는 후보는 정확도가 96.3%로 차이가 작으면서 계산은 평균 1.5회였기 때문에, 이 후보도 같이 비교했습니다.
  - 33만여 개 조합 중에서 고른 1등은 검증셋에 과적합됐을 수 있습니다. 그래서 후보 17개를 먼저 확정한 뒤에 테스트셋로 평가했습니다.

결과

- 테스트셋에서는 계산이 적어 같이 비교한 현재 문장 우선 Weighted Voting이 **91.9%**로 가장 높았고, 검증셋 1등은 90.6%였습니다. 그래서 이 방식을 앱에 적용했습니다.

표 2. 테스트셋 쿼리 선택 정확도 (160개)

방법	정답 수	정확도
고정 (현재 문장만 / 이전 대화 포함)	80/160	50.0%
Retrieval Score 분포	94/160	58.8%
Semantic Prototype 유사도	140/160	87.5%
Weighted Voting (검증셋 1등)	145/160	90.6%
현재 문장 우선 Weighted Voting	147/160	**91.9%**